Miłosz Kutyła

Application Security — laboratories
Topic #4: Authentication module

March 17, 2026

Contents

1. Introduction . . . 1
2. Module description . . . 2
2.1. Purpose . . . 2
2.2. Data collection . . . 2
2.3. Security assumptions . . . 2
3. Component requirements . . . 2
3.1. Functional requirements . . . 2
3.2. Non-functional requirements . . . 3
4. Component architecture . . . 4
5. Database structure . . . 5
6. UML sequence diagrams . . . 6
6.1. Registration sequence diagram . . . 6
6.2. Email activation sequence diagram . . . 7
6.3. Requesting a new activation email sequence diagram . . . 8
6.4. Authentication sequence diagram . . . 9
6.5. MFA enrollment sequence diagram . . . 10
6.6. MFA verification sequence diagram . . . 11
6.7. User password change sequence diagram . . . 12
6.8. User logout sequence diagram . . . 13
6.9. Password reset sequence diagram – request . . . 14
6.10. Password reset sequence diagram – password change . . . 15
7. Implementation . . . 16
7.1. User flow: registration . . . 16
7.2. User flow: activation . . . 18
7.3. User flow: first login . . . 19
7.4. User flow: forgotten password . . . 21
7.5. Details . . . 23
8. Conclusions . . . 28

1. Introduction

The goal of this lab was to extend the registration module, completed as part of a previous laboratory exercise, into a fully functional authentication module. This module should additionally cover: user login and logout (session disposal), server-side session management (or token-based session management), and a password reset ("forgotten password") feature. The website used for creating the UML diagrams was `https://mermaid.live/`.

2. Module description

2.1. Purpose

The authentication module handles user registration, user login, session management, and password reset processes:

- The user registration submodule serves as the initial gateway for user access, allowing new users to safely enroll in the system. I designed it to be a generic component that could later be expanded. Consequently, I chose email and username as the key identifiers for users within the system.
- The login submodule authenticates users and establishes secure sessions using cookies, while the MFA submodule adds an additional layer of security through time-based one-time passwords (TOTP).
- The password reset submodule allows users to securely recover access to their accounts when they forget their credentials.
- The logout submodule handles proper session termination and cleanup.

2.2. Data collection

The module is designed to collect the minimum amount of data necessary to establish and authenticate a unique identity:

- Email address (required): used as the unique login identifier and for communication with the user (e.g., sending the account activation link).
- Username (required): used as the unique public display name.
- Password (required): used for authentication.

2.3. Security assumptions

The following security assumptions are made for the module:

- The application operates over an encrypted channel (HTTPS) to prevent eavesdropping during data transmission.
- The database is not directly accessible from the public network.
- Server-side validation is the primary source of truth; client-side validation is only for User Experience (UX).
- A registered user must activate their account through an activation link and set up multi-factor authentication (MFA). The registration and system enrollment process is deemed complete once the user activates their account and sets up MFA.
- Users are required to pass reCAPTCHA verification on forms used for registration, login, password reset, and resending the activation email.
- In order to authenticate, the user is required to provide an email, password, and complete the MFA challenge.
- CSRF tokens are enforced for all sensitive endpoints (using POST, PUT, or DELETE methods).
- Rate limiting is implemented on all authentication endpoints to prevent brute-force attacks and abuse.
- Sessions are securely managed server-side with cryptographically secure session identifiers.

3. Component requirements

3.1. Functional requirements

This module has the following functional requirements:

1. Registration form: the system provides an interface for users to input their email, username, and password in order to register in the system.
2. Login form: the system provides an interface (or interfaces) for users to authenticate their email, username, password, and MFA code.
3. Logout functionality: the system provides a mechanism for users to terminate their session and clear the authentication state.
4. Input validation: the backend validates inputs against security rules (e.g., email format and uniqueness). This also includes enforcing password complexity policies and performing domain filtering by accepting or rejecting an email based on whether the domain part is restricted to the PUT domain (domain whitelisting).

5. reCAPTCHA: the system integrates Google reCAPTCHA (v2) on forms used for registration, login, password reset, and resending the activation email, to prevent automated attacks.
6. User data and password storage: successfully validated user data is stored in the database. Passwords are hashed and never stored in plain text.
7. Token generation and expiry: upon registration, the system generates a cryptographically secure, random activation token. This token must have a configurable expiration time (e.g., set to 15 minutes), after which the activation link becomes invalid.
8. Activation email: the system sends an email containing the activation link.
9. Account activation: accessing the activation link verifies the user and changes the user status from inactive to active (if accessed before the token expires).
10. Resending activation email: user should be able to request new activation link after expiry (or on delivery error) by providing the email used for registration.
11. MFA setup: upon first login after account activation, users are required to set up multi-factor authentication using TOTP with an authenticator app (e.g., Google Authenticator). The user is also provided with backup codes, in case the authenticator app is lost.
12. MFA enforcement: after MFA setup is complete, all subsequent login attempts must include a valid MFA code (or backup code) in addition to email and password.
13. Session management: the system creates and manages server-side sessions upon successful authentication, associating each session with a unique, cryptographically secure session identifier stored in an HTTP-only, secure cookie.
14. Password reset: users can request a password reset by providing their email address. The system sends a password reset link to the registered email if the account exists. Users can set a new password by accessing the reset link (before token expiration) and providing a new password that meets the complexity requirements.
15. Rate limiting: the system implements rate limiting on critical endpoints (registration, login, password reset request, resending activation link) to prevent brute-force attacks, credential stuffing, and denial-of-service attempts.
16. Session invalidation on logout: when a user logs out, their session is immediately invalidated on the server, and the session cookie is cleared from the client.

3.2. Non-functional requirements

The registration module has the following non-functional requirements:

1. Password storage security: passwords must be hashed using a strong algorithm (PBKDF2) with a unique salt for each user.
2. Responsiveness: the frontend must provide immediate visual feedback on the registration status. The system must provide clear, generic error messages that do not expose internal implementation details to the user.
3. Modularity: the registration logic should be decoupled from the main business logic so that it is generic and can be reused and expanded.
4. Performance: the time taken for the server to process the authentication requests (excluding network latency and email sending) should not exceed a specified threshold (e.g., 500 ms).
5. Availability: authentication endpoints must maintain high availability even under attack.
6. Data in transit: all data transmitted between the client and the server must be secured by enforcing HTTPS.
7. Security event logging: the system must record specific security events (e.g., failed registration, token usage).
8. Session security: session identifiers must be cryptographically random, stored in HTTP-only and secure cookies.
9. Rate limit configuration: rate limits should be configurable per endpoint with appropriate thresholds.

4. Component architecture

The system follows a client-server architecture, communicating via a RESTful API:

- Frontend: implemented in React.js. It handles the presentation layer, form rendering, provide immediate local form validation, and display appropriate messages based on backend's responses.
- Backend: implemented in Python with Django. This layer exposes API endpoints, handles business logic, server-side validations, password hashing, token generation, user creation, and account activation.
- Database: relational SQLite3 database storing user identities and tokens.
- SMTP server: PUT SMTP server `poczta.student.put.poznan.pl` using implicit SSL/TLS over port 465; it is used for sending activation emails.
- Google reCAPTCHA: frontend collects the reCAPTCHA token, and the backend verifies it with Google's reCAPTCHA API before processing the request.
- Deployment: the built frontend is served via nginx, which also serves as a reverse proxy for the Django backend. HTTPS is enforced for all external traffic and terminated at nginx. This termination point is chosen because the entire stack is hosted on a single server, meaning that internal communication occurs over the local loopback interface. Note: this choice was made to allow inspecting the internal HTTP traffic for demonstration and debugging, accepting the risk of sniffing the unencrypted internal traffic.

  The diagram of the designed system architecture is shown in Figure 1.

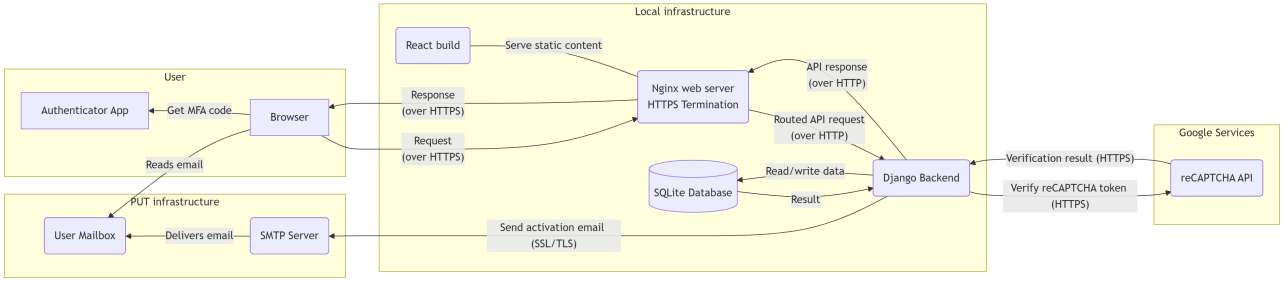


Figure 1: System architecture diagram

5. Database structure

The system utilizes a relational database to store user identities. The schema for the Users table (used to store user identities and tokens) is presented in Table 1.

Table 1: Structure of the User table

Field name	**Type**	**Constraints**	**Description**
`id`	Integer	PK, Auto Increment	Internal unique identifier.
`email`	Varchar(254)	Unique, Not Null	User's email, primary user identifier.
`password`	Varchar(128)	Not Null	Password string: algorithm, salt, hash.
`username`	Varchar(30)	Unique, Not Null	Public display name.
`date_joined`	DateTime	Not Null	Timestamp of registration.
`last_login`	DateTime	Nullable	Timestamp of last login.
`is_active`	Boolean	Default: False	Determines if the user can log in.
`is_staff`	Boolean	Default: False	Determines if the user can log in.
`is_superuser`	Boolean	Default: False	Determines if the user can log in.
`action_token_hash`	Varchar(64)	Unique, Nullable	Action token SHA256 hash.
`action_token_type`	Varchar(32)	Nullable	Action token type: email verification, password reset (...)
`token_token_expiry`	DateTime	Nullable	Timestamp when the token becomes invalid.
`mfa_secret`	Varchar(64)	Nullable	MFA secret.
`mfa_enabled`	Boolean	Default: False	Flag indicating whether user has MFA set up.
`mfa_backup_codes`	List	Default: False	List of backup codes' hashes.

The schema for the Sessions table (used to store active sessions) is presented in Table 2.

Table 2: Structure of the Sessions table

Field name	**Type**	**Constraints**	**Description**
`session_key`	Varchar(40)	PK, Not Null	Unique session identifier.
`session_data`	Text	Not Null	Serialized data associated with the session.
`expire_date`	Datetime	Not Null, Indexed	Expiry date of the session.

6. UML sequence diagrams

6.1. Registration sequence diagram

The UML sequence diagram for user registration is shown in Figure 2.

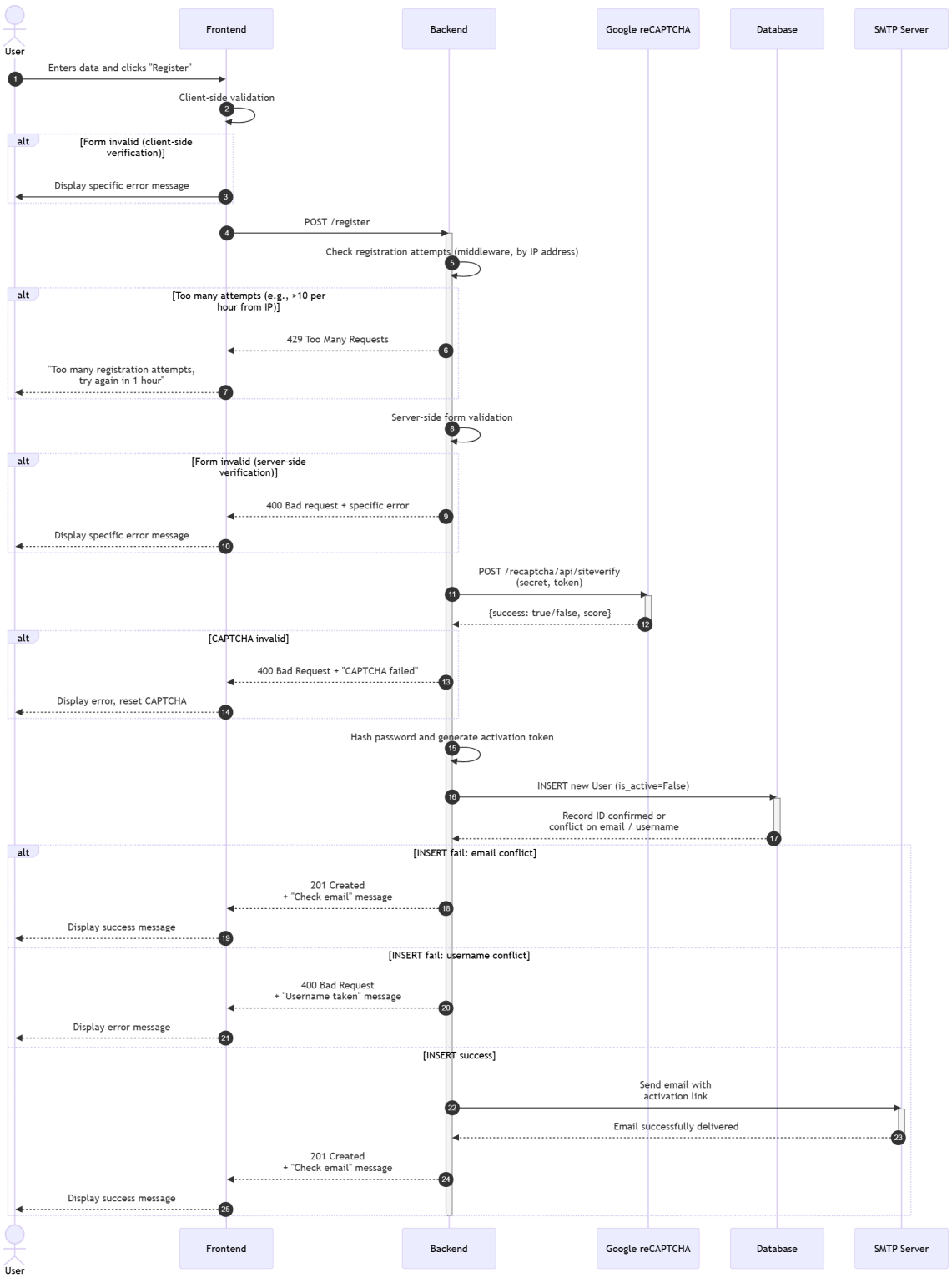


Figure 2: UML sequence diagram for user registration

6.2. Email activation sequence diagram

The UML sequence diagram for user account activation is shown in Figure 3.

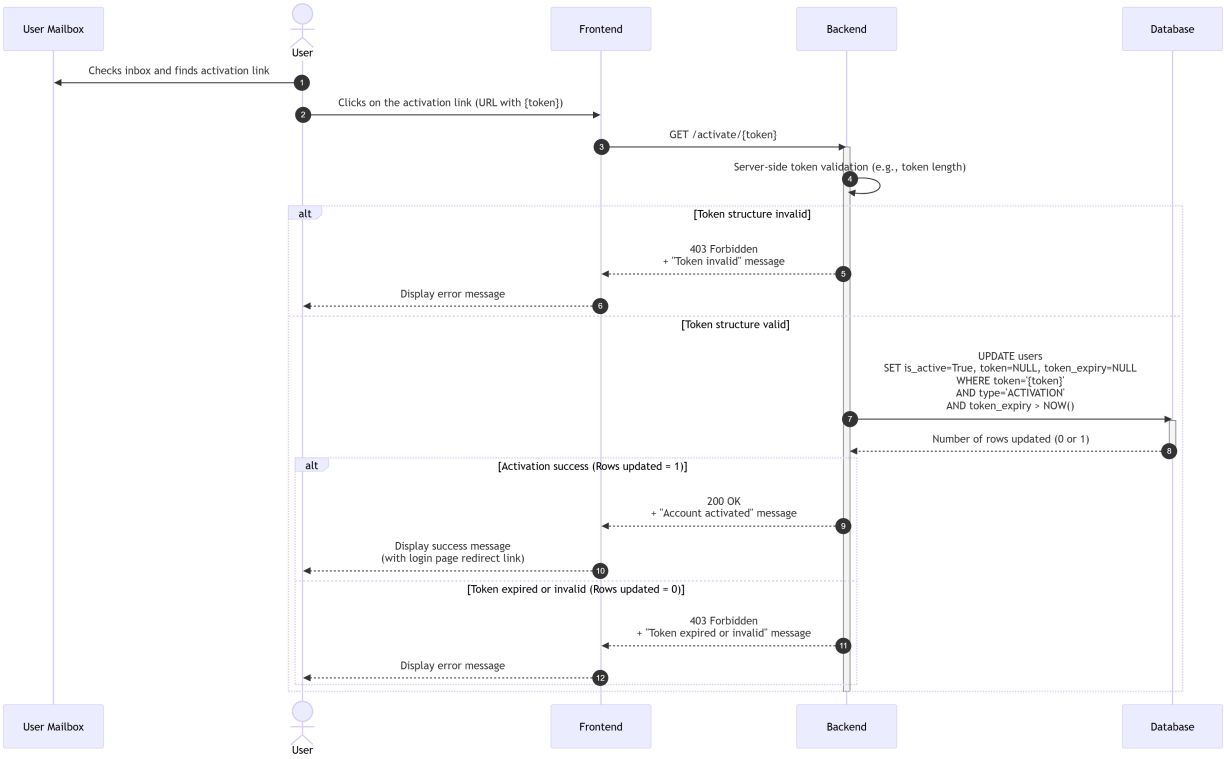


Figure 3: UML sequence diagram for user account activation

6.3. Requesting a new activation email sequence diagram

The UML sequence diagram for requesting a new activation email is shown in Figure 4.

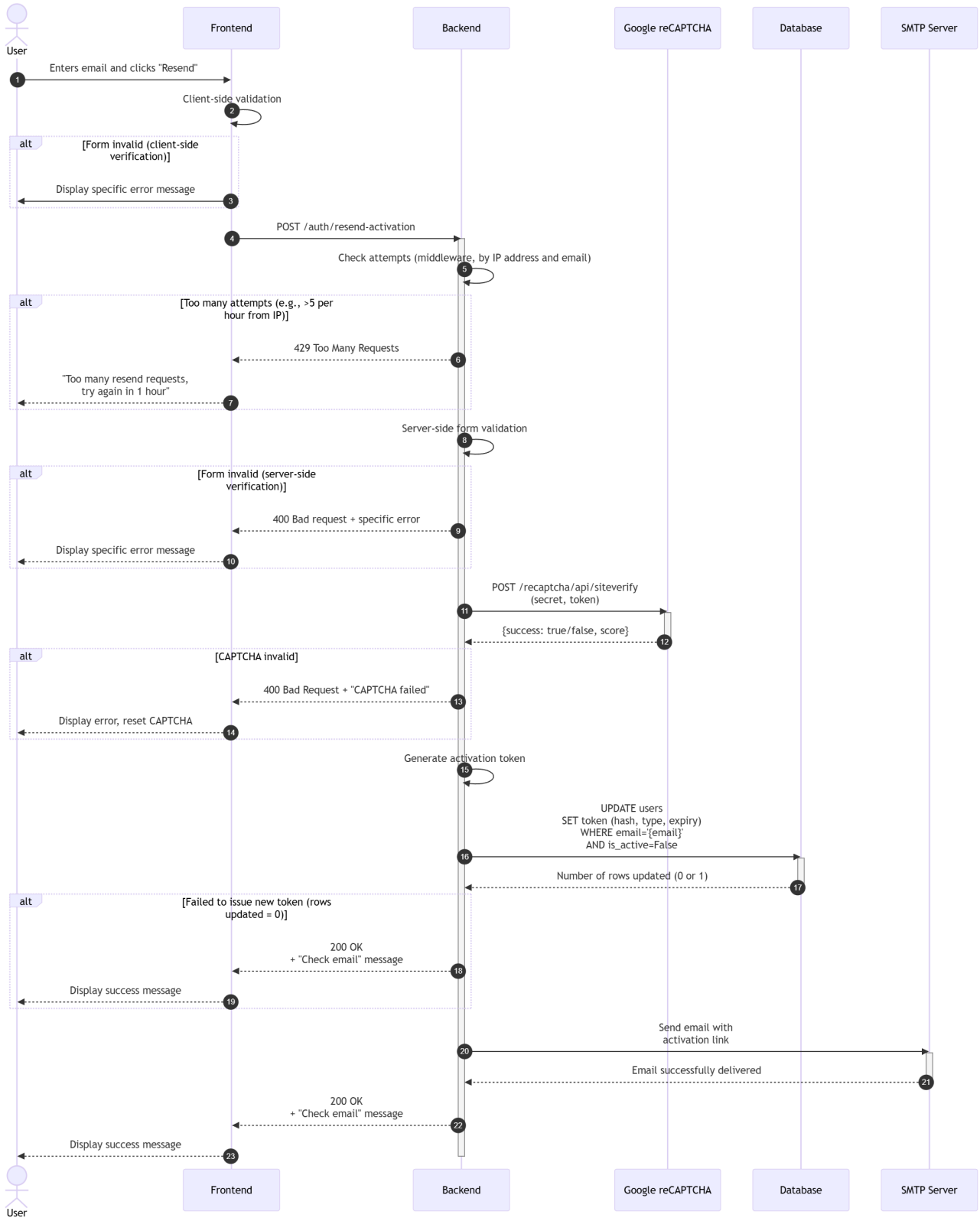


Figure 4: UML sequence diagram for requesting a new activation email

6.4. Authentication sequence diagram

The UML sequence diagram for user authentication is shown in Figure 5.

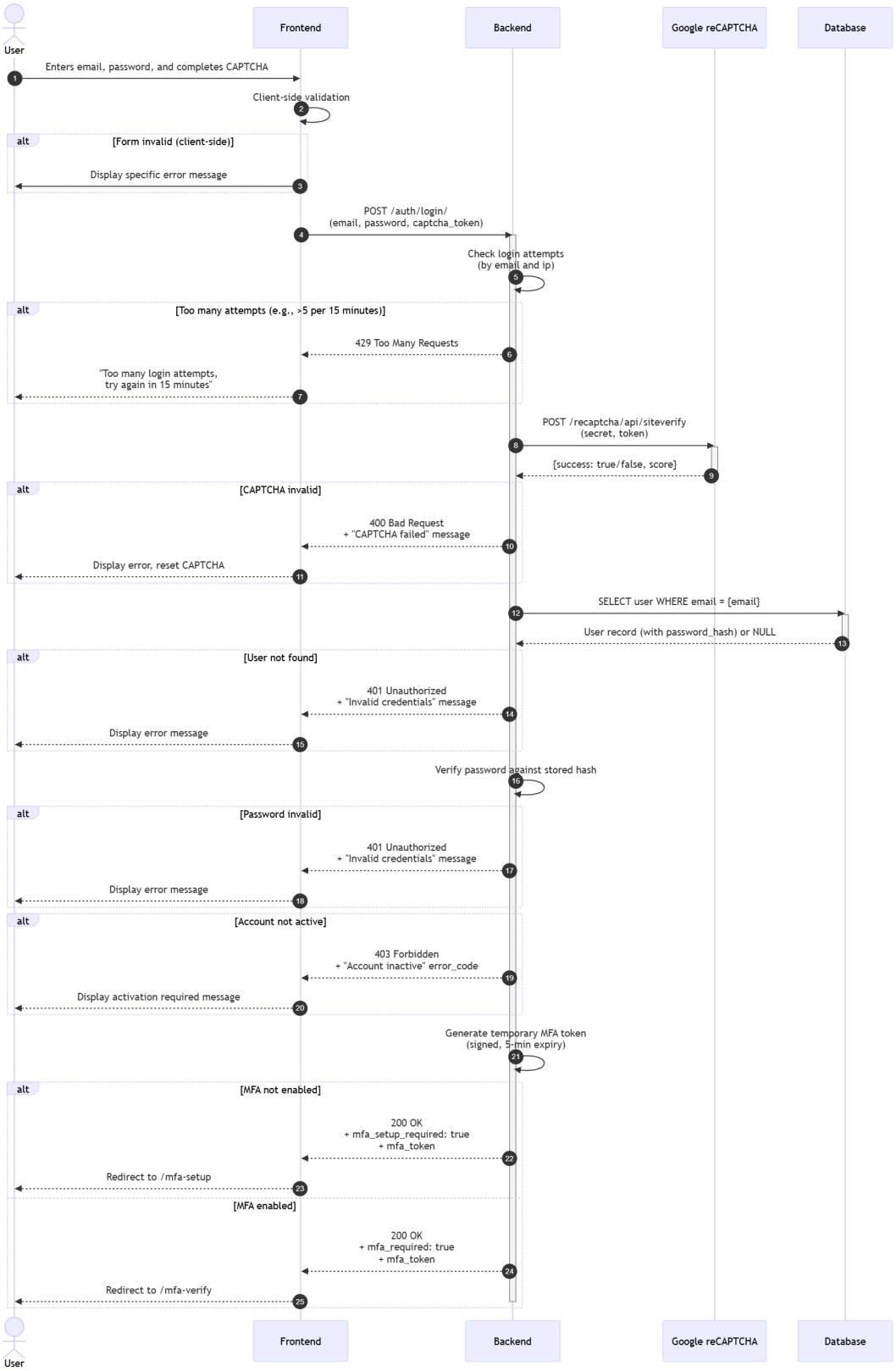


Figure 5: UML sequence diagram for user authentication

6.5. MFA enrollment sequence diagram

The UML sequence diagram for user MFA enrollment is shown in Figure 6.

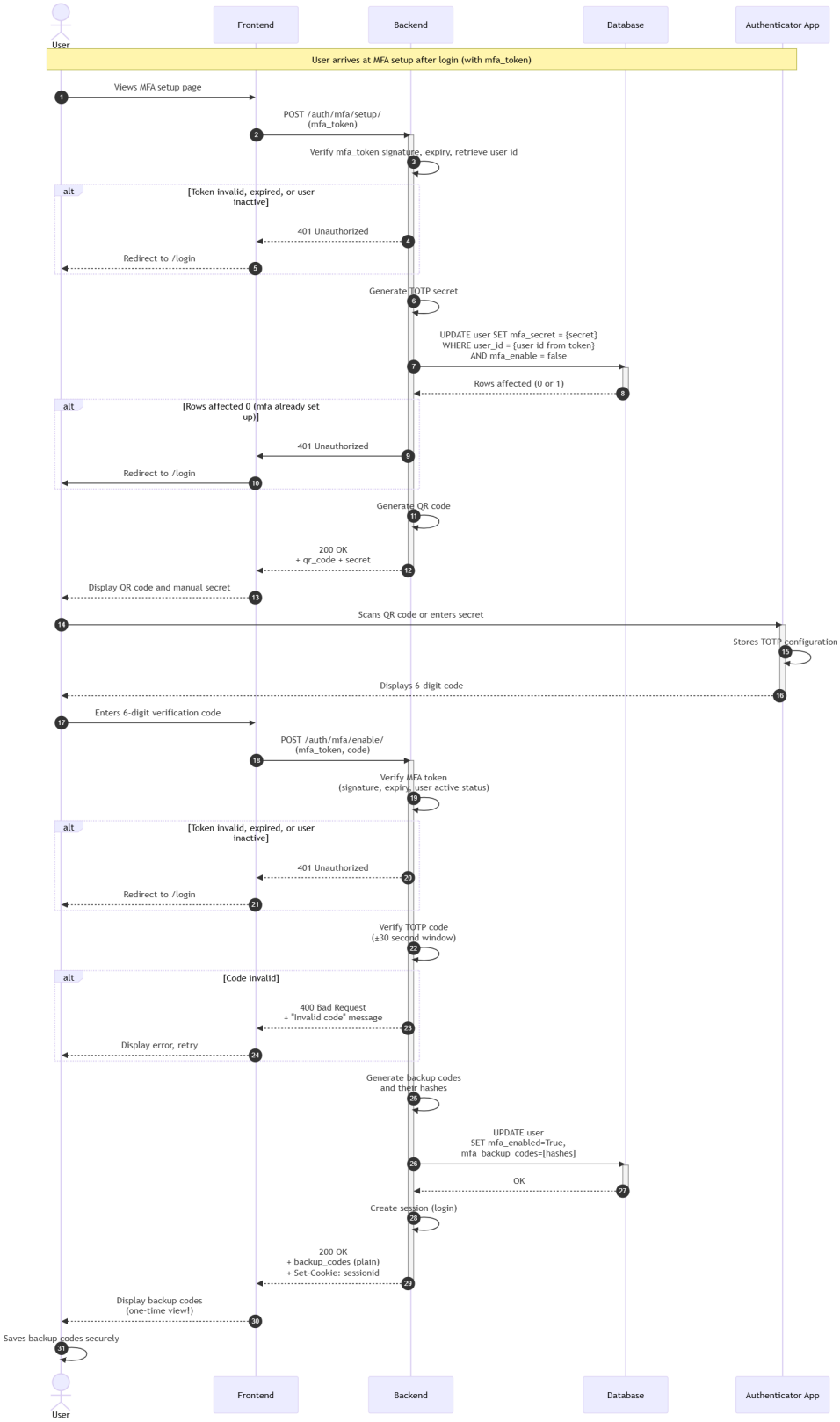


Figure 6: UML sequence diagram for user MFA enrollment

6.6. MFA verification sequence diagram

The UML sequence diagram for user MFA verification is shown in Figure 7.

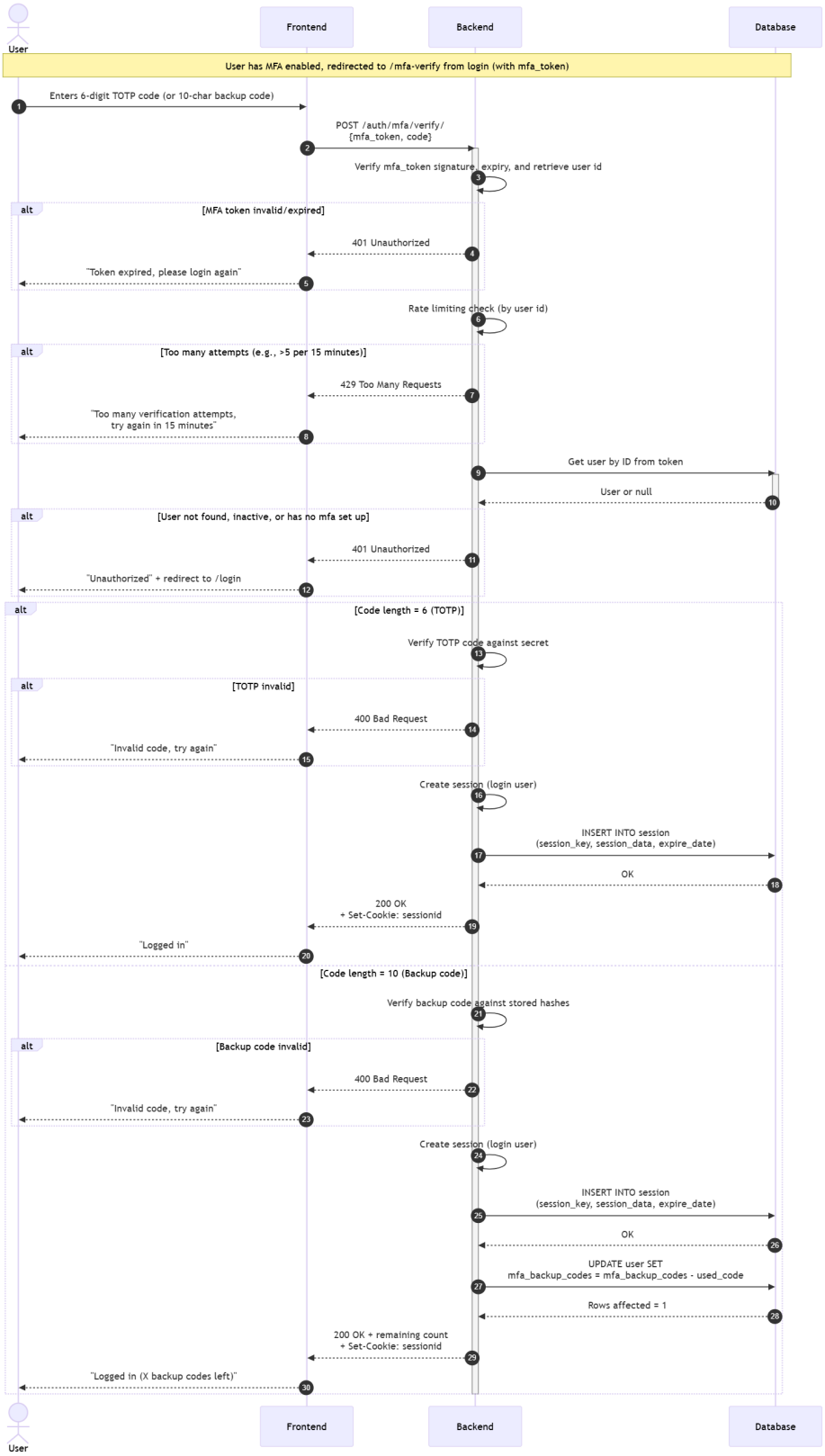


Figure 7: UML sequence diagram for user MFA verification

6.7. User password change sequence diagram

The UML sequence diagram for the user password change is shown in Figure 8.

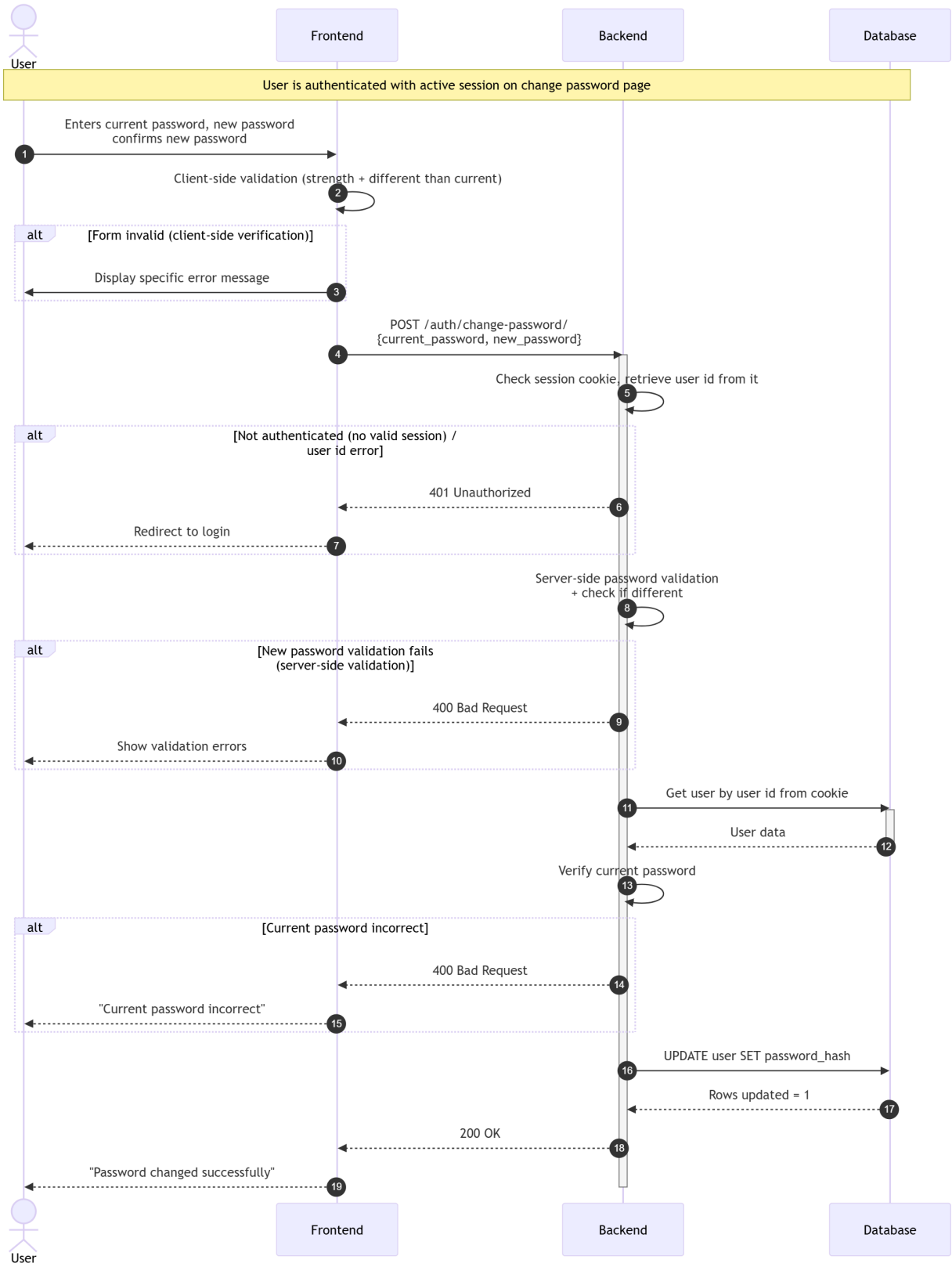


Figure 8: UML sequence diagram for the user password change

6.8. User logout sequence diagram

The UML sequence diagram for user logout is shown in Figure 9.

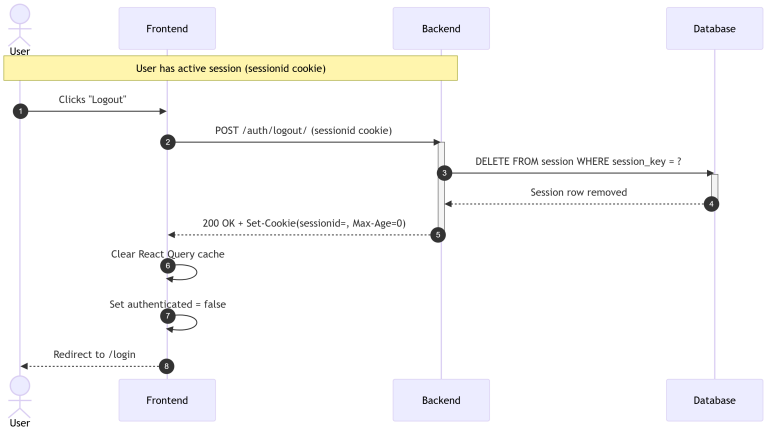


Figure 9: UML sequence diagram for user logout

6.9. Password reset sequence diagram – request

The UML sequence diagram for password reset (request) is shown in Figure 10.

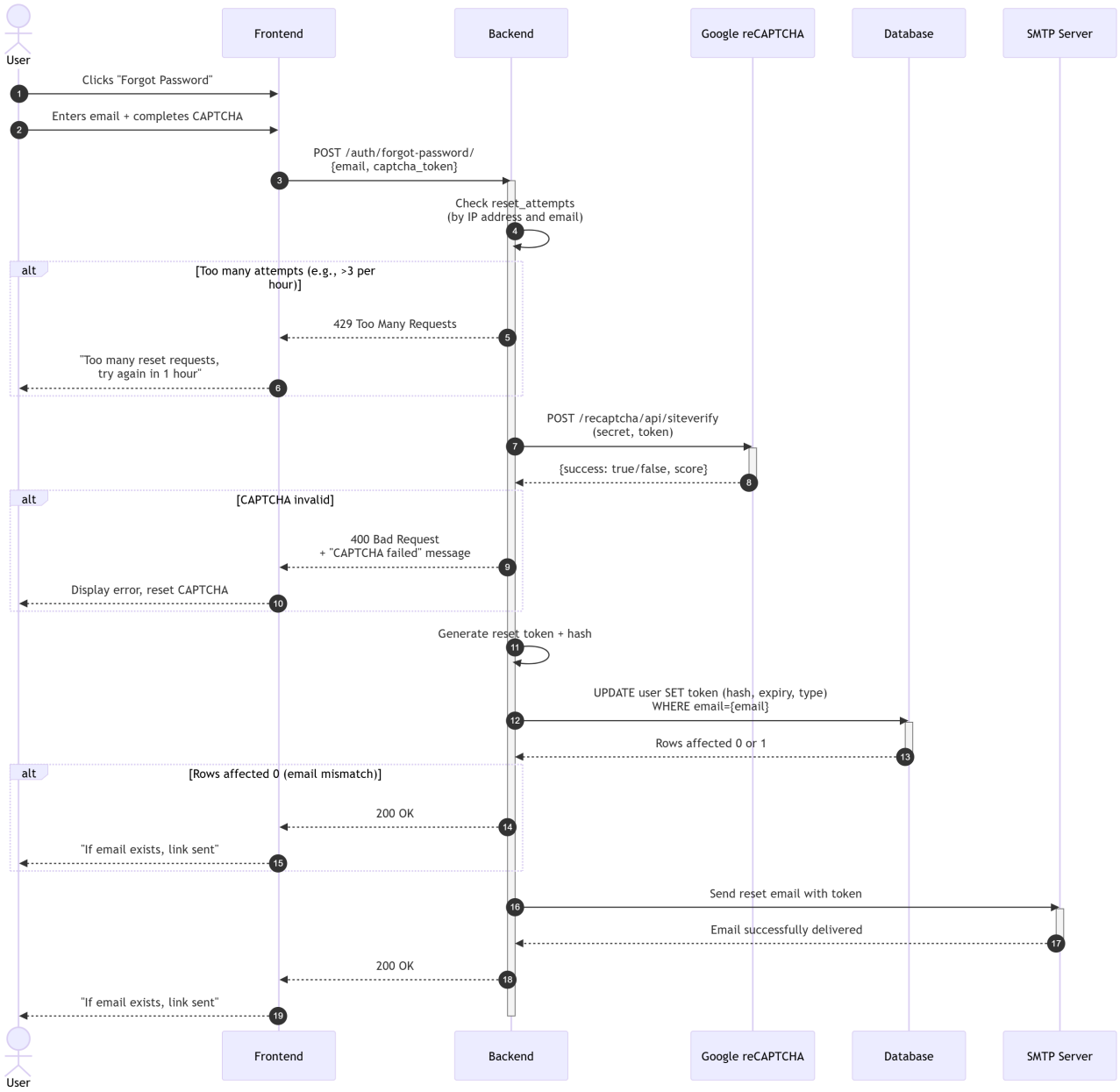


Figure 10: UML sequence diagram for password reset request

6.10. Password reset sequence diagram – password change

The UML sequence diagram for password reset (changing the password) is shown in Figure 11.

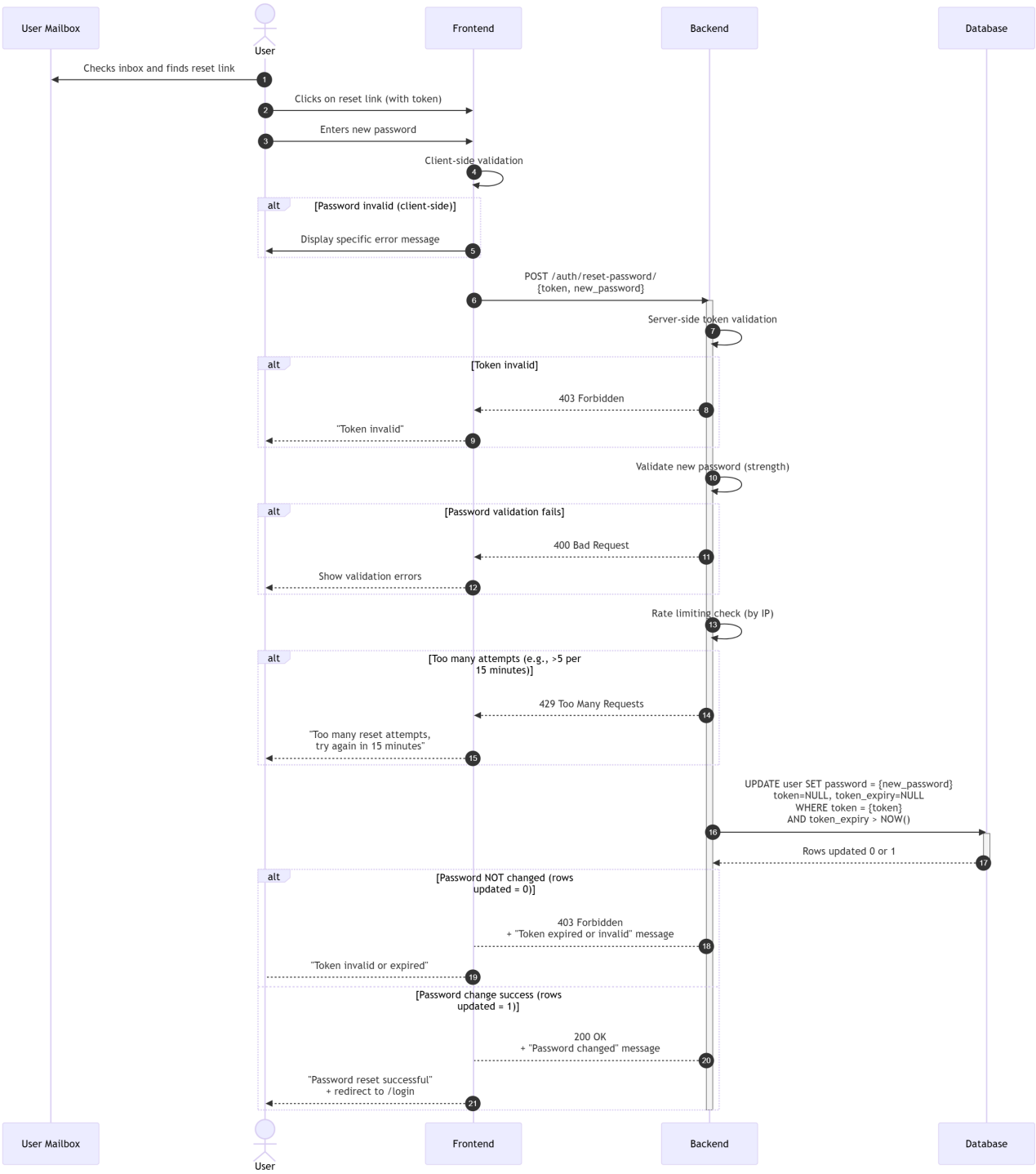


Figure 11: UML sequence diagram for password change after requested reset

7. Implementation

7.1. User flow: registration

My registration form requires four inputs: email (restricted to `the put.poznan.pl` domain), username (5-30 alphanumeric characters), password (at least 12 characters long, containing at least one uppercase letter, one lowercase letter, one number, and a special character), and password confirmation (for user experience).

While the user fills in the form, the frontend tracks which requirements are met or not met and displays their status for the user to monitor. For example, if the email, username, and password requirements are not met, as shown in Figure 12, specific requirements are highlighted in red (not met), green (met), or gray (cannot be applied yet: for example, when the password confirmation field is empty). As long as not every requirement is met, the page does not allow the form to be submitted. Once all requirements are met, the form can be submitted, as shown in Figure 13.

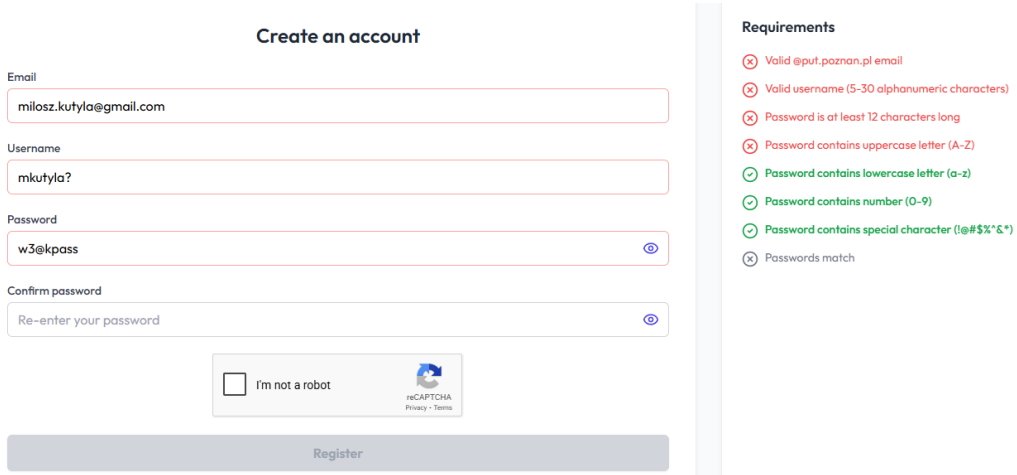


Figure 12: Registration form client-side verification: invalid email, username, password

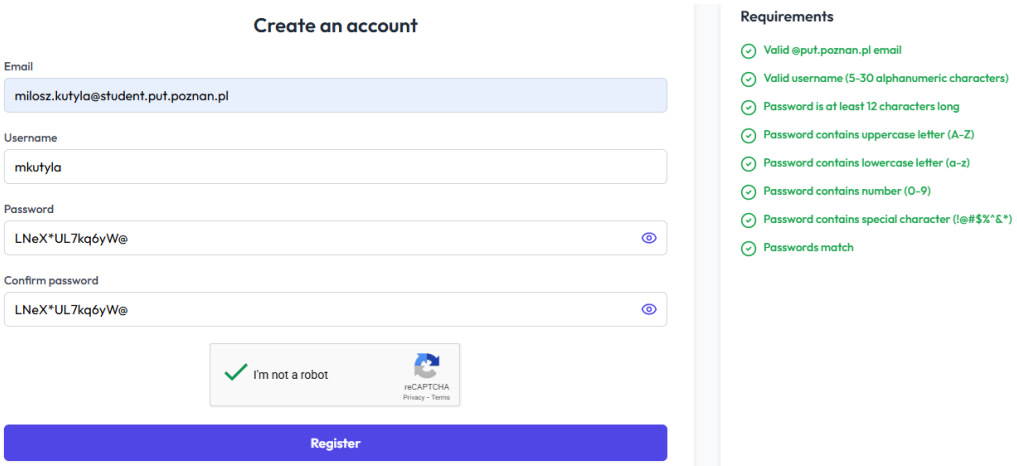


Figure 13: Registration form client-side verification: valid form

The form is also validated server-side. Once it is submitted, the backend checks for the uniqueness of the email and username. If a conflict occurs, an HTTP 409 response with an appropriate message is returned. The frontend displays a message based on the response received, as shown for the username conflict in Figure 14.

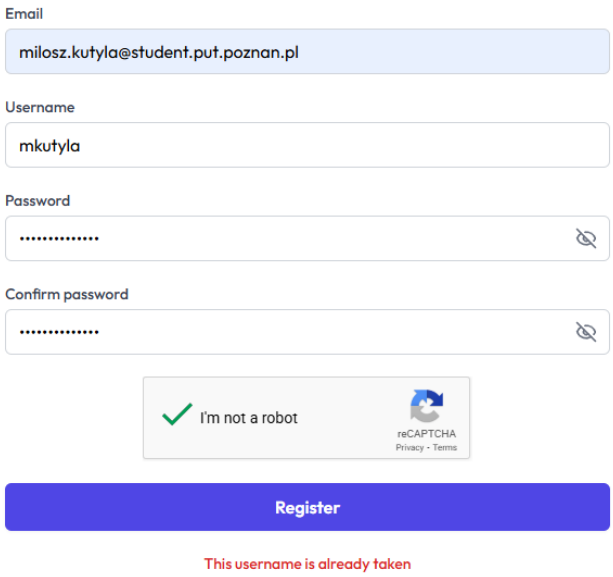


Figure 14: Invalid form: username taken (server-side validation)

If the data sent to the backend is validated successfully, an activation email is sent, an HTTP 201 response is returned, and the frontend redirects to a success page directing the user to check their email, as shown in Figure 15. This page is also displayed in the case of an email conflict, but the activation email is not sent in this situation. Backend responding with a 201 in this case prevents information disclosure.

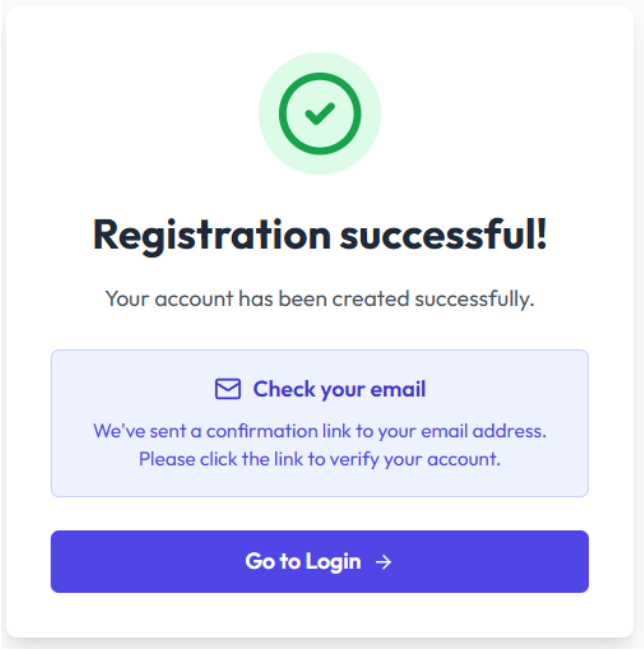
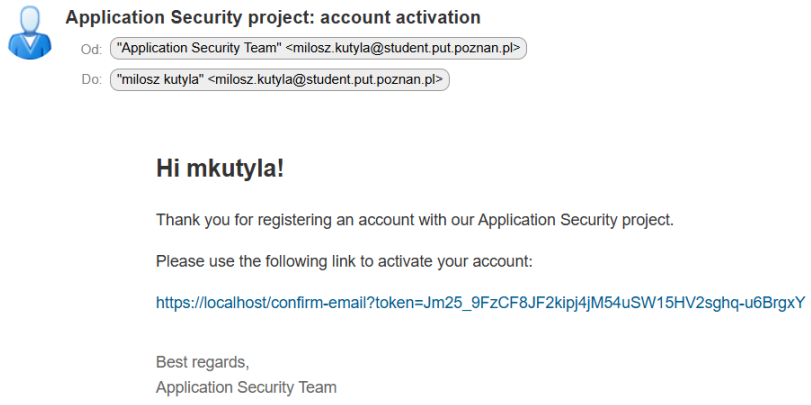


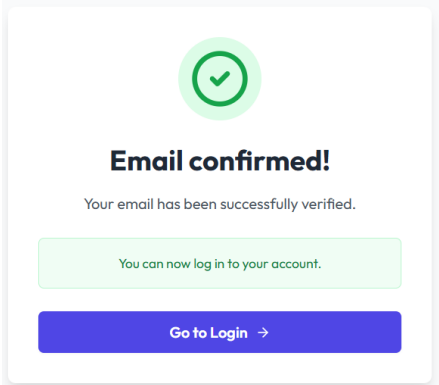
Figure 15: Successful registration

7.2. User flow: activation

Once successfully registered, the user can check their mailbox and activate their account using the activation link. I use my own student account to send the emails from the backend with an "Application Security Team" display name, as shown in the received email message in Figure 16a. Once the user clicks on the link, a request is made to the backend (with the activation token), activating their account, as shown in Figure 16b.



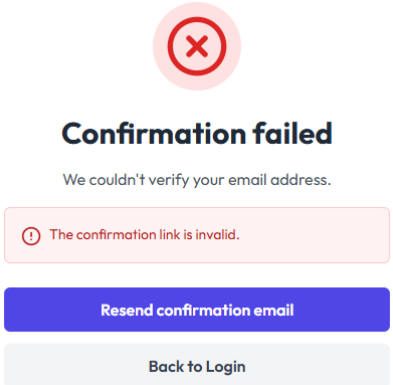
(a) Activation email sent to my mailbox



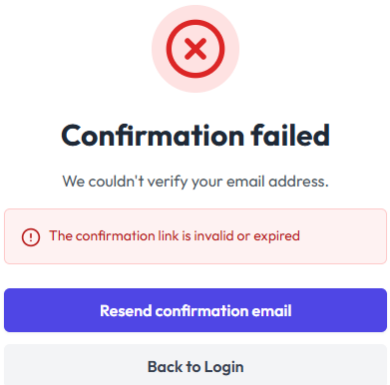
(b) Email confirmed successfully

Figure 16: Account activation

Of course, the activation link can be invalid or expired, which results in the frontend displaying the "Confirmation failed" page. Depending on the reason for not accepting the token, the frontend displays different information (based on the backend's response). If the token has an invalid format (here, I tampered with its length), an appropriate message is displayed, as shown in Figure 17a. Similarly, if the token is either reused, expired, or invalid, but it has a valid format, an ambiguous message is displayed (i.e., not leaking whether the token was ever issued or not), as shown in Figure 17b.



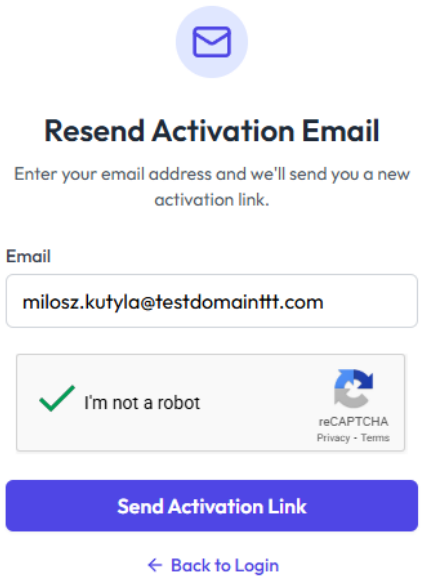
(a) Invalid token structure



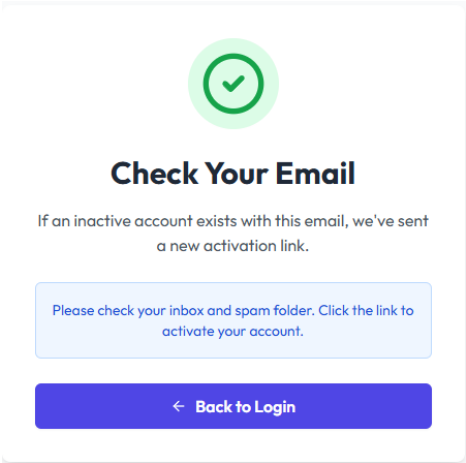
(b) Reused, expired, or invalid token (valid structure)

Figure 17: Tampering with the activation link: failed email confirmation attempts

The user can request a new activation email after clicking the "Resend confirmation email" and inserting an email as shown in Figure 18a. If an inactive account with such email exists in the system, a new activation email will be sent, as shown in Figure 18b.



(a) Resending confirmation email form

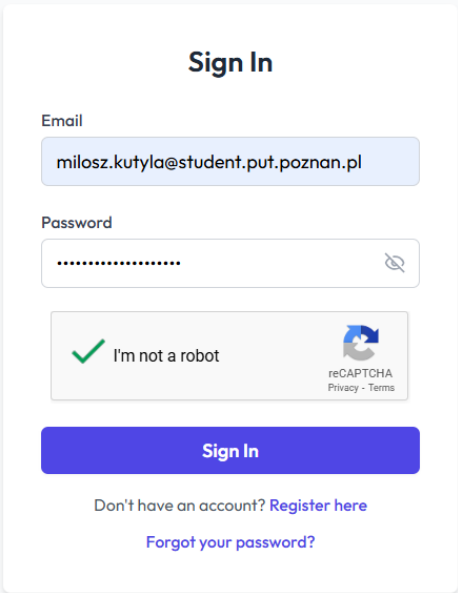


(b) Message after requesting a new activation email

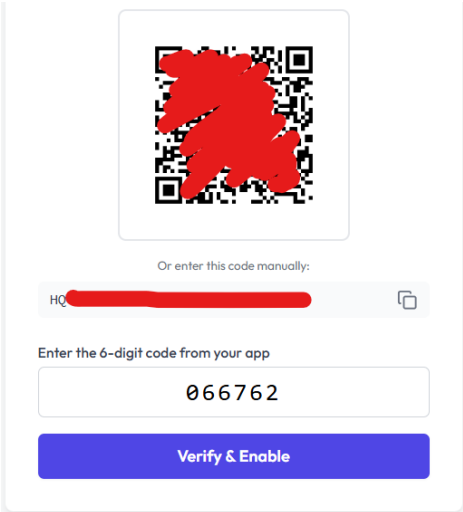
Figure 18: Resending the activation email

7.3. User flow: first login

Once the account is activated, the user can log in for the first time using the login form show in Figure 19a. After successfully logging in, the user is required to set up 2FA, as shown in Figure 19b.



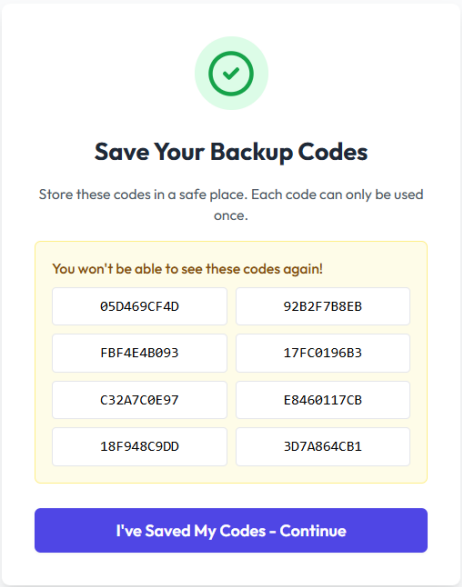
(a) Login form



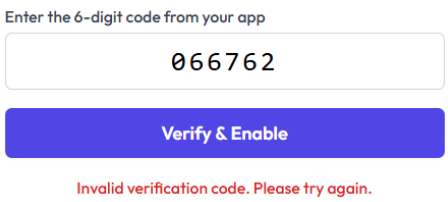
(b) QR code / secret to enter in the authenticator app

Figure 19: First login and setting up 2FA

If the code entered in the box is valid, the user is presented with backup codes as shown in Figure 20a. If it's invalid, the page displays an error as shown in Figure 20b.



(a) Backup codes for 2FA



(b) Entering invalid code

Figure 20: Finishing setting up 2FA

When logging in next time, the user is required to enter a code from their authenticator app as shown in Figure 21.

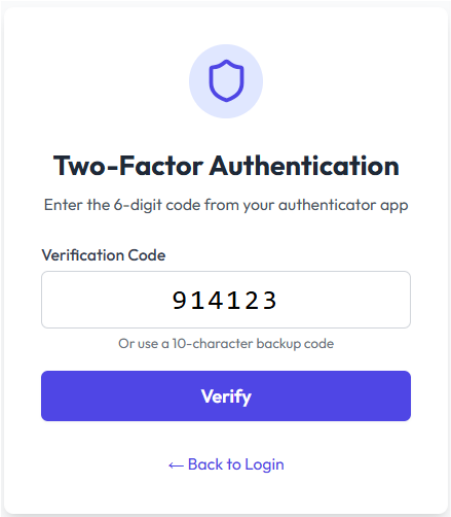
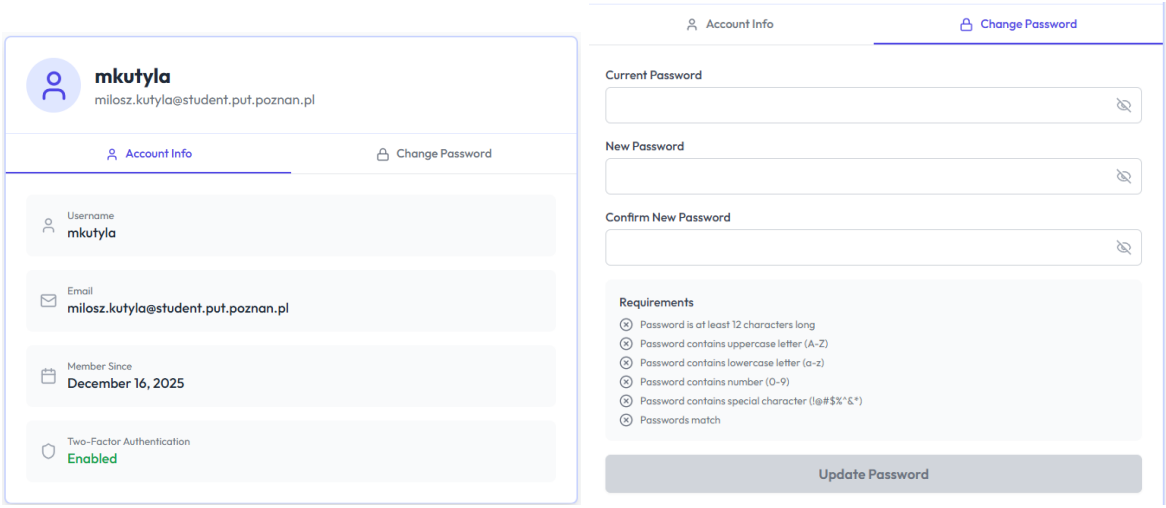


Figure 21: 2FA on next login attempts

After this the user is authenticated they can view their profile and change their password, as shown in Figures 22a and 22b.

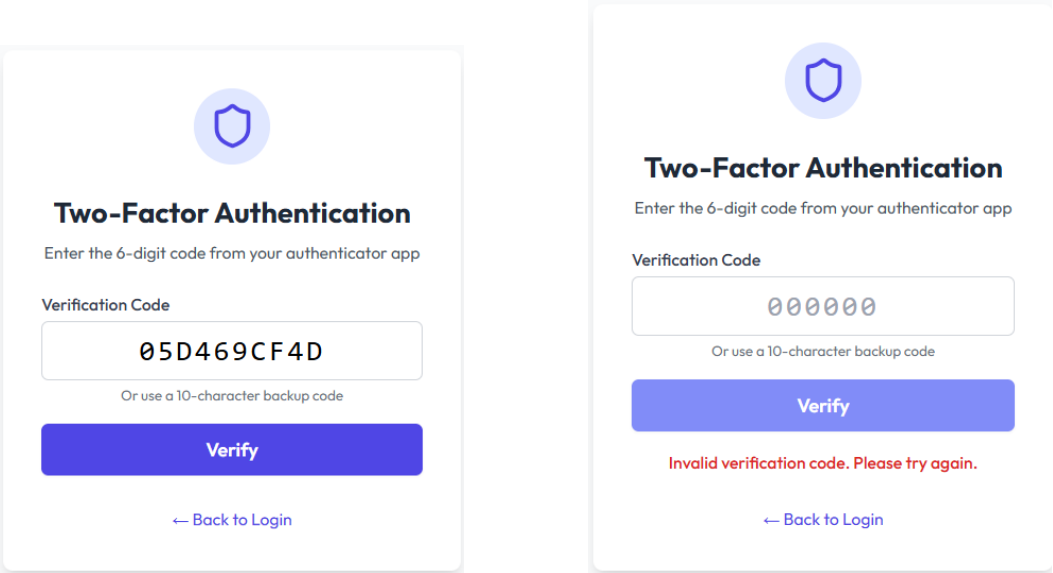


(a) User account details

(b) Password change form

Figure 22: User profile

Alternatively, user can enter a backup code as shown in Figure 23a. If user enter an invalid or expired code, an error is displayed as shown in Figure 23b.



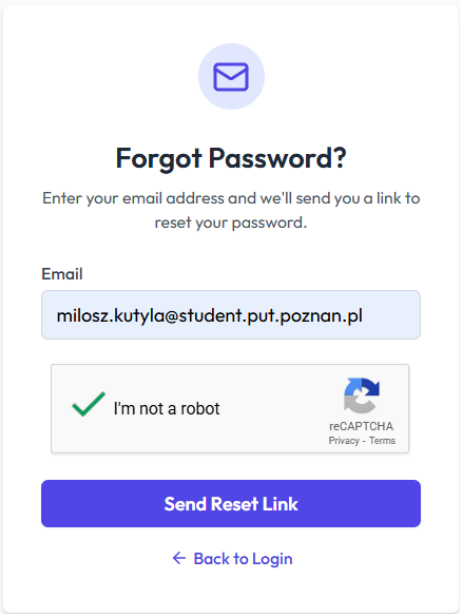
(a) Using the backup code

(b) Entering an invalid or already used code

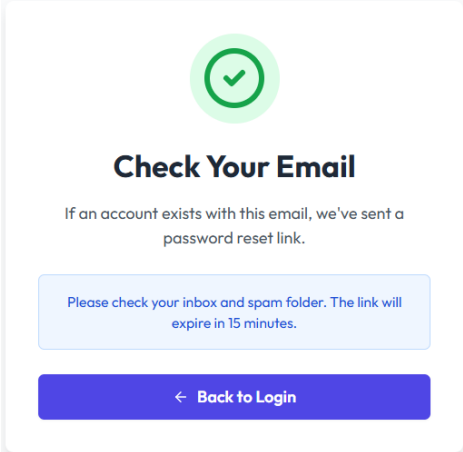
Figure 23: 2FA with backup codes

7.4. User flow: forgotten password

If user forgets their password, a forgotten password functionality can be used. After entering the forgotten password page, the user is required to enter an email to their account as shown in Figure 24a. If it exists, an email will be sent, as shown in Figure 24b.



(a) Making the password reset request



(b) Message about email being sent

Figure 24: Forgotten password functionality

If an account with matching email exists then an email is actually sent, as shown in Figure 25.

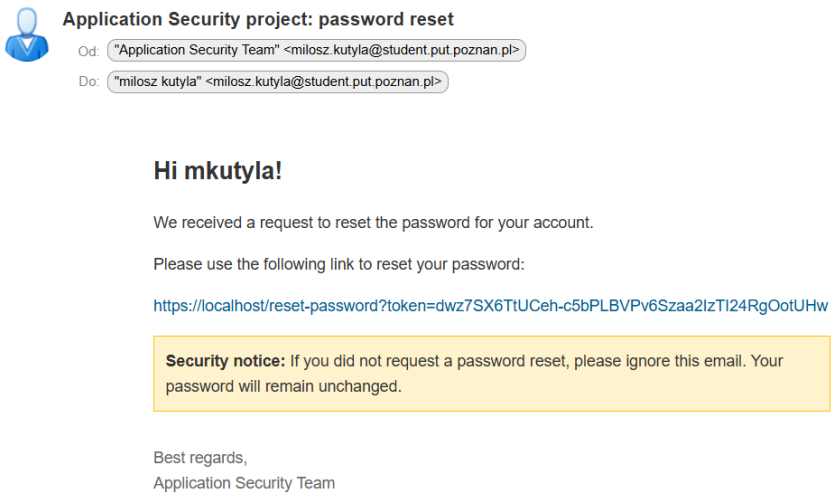
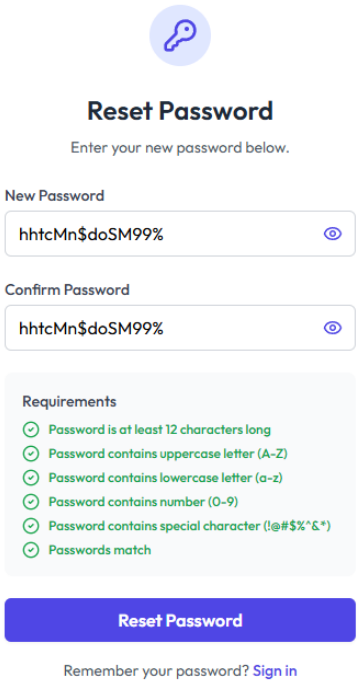
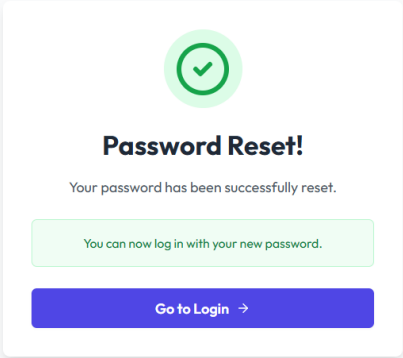


Figure 25: Email with the password reset link

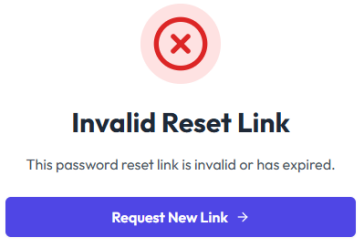
After clicking the link, the user is redirected to password reset (change) page as shown in Figure 26a where the password can be changed. After passing a valid password (i.e., fulfilling complexity requirements) the user is notified about successful password change as shown in Figure 26b. If the user tampers with the link or uses it after it expires, an error is displayed as shown in Figure 26c.



(a) Making the password reset request



(b) Message about password change



(c) Messaged displayed on invalid or expired password change link

Figure 26: Password change operations

7.5. Details

7.5.1. Validation

There are several checks for the form fields:

- Email must come from the `put.poznan.pl` domain. Both frontend and backend perform a check against the exact same regex pattern.
- Username must be 5 to 30 characters long and contain only alphanumeric characters. Both frontend and backend perform a check against the exact same regex pattern.
- Password must comply with the following requirements:
  - ◇ it must be at least 12 characters long,
  - ◇ it must contain at least one uppercase letter,
  - ◇ it must contain at least one lowercase letter,
  - ◇ it must contain at least one number,
  - ◇ it must contain at least one special character,
  - ◇ it must not be similar to the email or username (Django's `UserAttributeSimilarityValidator`),
  - ◇ it must not be a common password (Django's `CommonPasswordValidator`).

If any of these requirements are not met during server-side validation, the backend responds with a code 400 and an appropriate message indicating which validation failed.

The same password checks are enforced on the password reset and change forms (both client- and server-side). Similarly, the frontend performs validation of the email and username so that users do not send login requests with obvious typos.

7.5.2. Passwords storage

I store passwords in a hashed form using an advanced hashing algorithm: `pbkdf2_sha256` with 1,000,000 iterations and a unique salt per user. All password data is stored in a single field in the database.

7.5.3. Tokens

My approach to designing the tokens was made with consideration for hardening them. The tokens are generated using cryptographically secure random bytes (`secrets.token_bytes(32)`) and encoded in URL-safe Base64 format, resulting in a 43-character token with high entropy (256 bits). The tokens are never stored in plaintext in the database; instead, only their SHA-256 hashes are stored. This ensures that even if the database is compromised, attackers cannot use the stored hashes to activate accounts or reset passwords, as the tokens themselves remain unknown. Additionally, each token has an expiry time, limiting the window of opportunity for potential attacks. The action type of each token is stored separately, allowing the same token mechanism to be reused for different purposes (email verification, password reset, etc.) while maintaining a clear separation of concerns.

7.5.4. Database state updates

Once the user registers, an appropriate record is created in the database, as shown in Figure 27. We can see the action token's hash, type, and expiry date. After the user confirms their email, token-related fields are set to null, and the active status is set to true, as shown in Figure 28.

EMAIL	USERNAME	IS STAFF	IS ACTIVE	MFA ENABLED	ACTION TOKEN HASH	ACTION TOKEN TYPE
admin@put.poznan.pl	admin	✓	✓	✗	-	-
milosz.kutyla@student.put.poznan.pl	mkutyla	✗	✗	✗	b2671e27ffc0424c4fe14d31e40f89ac7a705c967e5719d49a23ee8be57fd633	EMAIL_VERIFICATION

Figure 27: Record before clicking the activation link

EMAIL	USERNAME	IS STAFF	IS ACTIVE	MFA ENABLED	ACTION TOKEN HASH	ACTION TOKEN TYPE	ACTION TOKEN EXPIRY
admin@put.poznan.pl	admin	✓	✓	✗	-	-	-
milosz.kutyla@student.put.poznan.pl	mkutyla	✗	✓	✗	-	-	-

Figure 28: Record after clicking the activation link

After logging in and setting MFA, the record has `mfa_enabled` flag set to true, as shown in Figure 29.

EMAIL	USERNAME	IS STAFF	IS ACTIVE	MFA ENABLED	ACTION TOKEN HASH	ACTION TOKEN TYPE	ACTION TOKEN EXPIRY
admin@put.poznan.pl	admin	✓	✓	✗	-	-	-
milosz.kutyla@student.put.poznan.pl	mkutyla	✗	✓	✓	-	-	-

Figure 29: Record after setting up MFA

When user tries to use the forgotten password functionality, we can see a new `PASSWORD_RESET` token as shown in Figure 30.

EMAIL	USERNAME	IS STAFF	IS ACTIVE	MFA ENABLED	ACTION TOKEN HASH	ACTION TOKEN TYPE
admin@put.poznan.pl	admin	✓	✓	✗	-	-
milosz.kutyla@student.put.poznan.pl	mkutyla	✗	✓	✓	53fe3dcd5dd252b576dc4a5d777006b6c01c95874ec90c88edbb30c1e2b4f9e4	PASSWORD_RESET

Figure 30: Record after using the forgotten password functionality

Authenticated users also have sessions stored in the database as shown in Figure 31.

session_key	session_data	expire_date
q0jlri1inugsk8qitozzfygj74etxol8	.eJxVjEEOwiAQRe_C2hDa4kxx6d4zkGEYpGogKe3KeHdD...	2025-12-14 10:33:01.623905

Figure 31: Session record

As designed and shown in the UML diagrams in Figures 2 and 3, all database operations are minimized. User creation and activation require only one database query each, and the response is based on whether they were successful, rather than querying the database for a result, performing checks on the data, and then querying the database again.

This approach is demonstrated in Figure 32, where a single UPDATE query finds a user based on their token hash, validates the token's action type and expiry date, and updates the user's status while invalidating the token only if all conditions are met. This atomic operation eliminates the need for separate SELECT and UPDATE queries, reducing database load. A similar approach is implemented for each database operation.

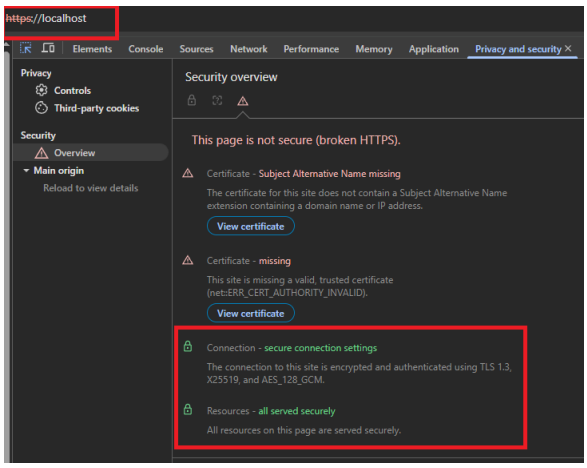
```python
def activate_user_and_invalidate_token(self, token_string: str):
    now = timezone.now()
    token_hash = hash_token(token_string)
    expected_action = ActionType.EMAIL_VERIFICATION

    updated_count = self.filter(
        action_token_hash__exact=token_hash,
        action_token_expiry__gt=now,
        action_token_type__exact=str(expected_action),
        is_active=False,
    ).update(
        is_active=True,
        action_token_hash=None,
        action_token_type=None,
        action_token_expiry=None,
    )
```

Figure 32: Python single-query account activation implementation

7.5.5. Enforcing HTTPS

Additionally, I enforce HTTPS, as shown in Figure 33a. I created a self-signed certificate and configured nginx to redirect all HTTP traffic to HTTPS, as shown in Figure 33b. The configuration uses TLS 1.2 and TLS 1.3 protocols with strong cipher suites. A 2048-bit Diffie-Hellman parameter file is used to enhance forward secrecy.



(a) HTTPS in browser

```
root@asubuntu:/etc/nginx/sites-enabled# cat appsec-pro
server {
        listen 80;
        listen [::]:80;
        server_name _;

        return 301 https://$host$request_uri;
}

server {
        listen 443 ssl;
        listen [::]:443 ssl;
        server_name _;

        ssl_certificate /etc/ssl/certs/nginx-selfsigned.crt;
        ssl_certificate_key /etc/ssl/private/nginx-selfsigned.key;
        ssl_dhparam /etc/ssl/certs/dhparam.pem;

        ssl_protocols TLSv1.3 TLSv1.2;
        ssl_prefer_server_ciphers off;
        ssl_ciphers 'TLS_AES_256_GCM_SHA384:TLS_CHACHA20_POLY1305_S
305:ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256';
```

(b) Nginx configuration enforcing HTTPS

Figure 33: Enforcing HTTPS

7.5.6. MFA

I implemented MFA using `pyotp` for TOTP generation and verification, `qrcode` for QR code generation, and `itsdangerous` for secure temporary MFA token creation. The system enforces mandatory two-factor authentication for all users, generating a QR code during setup that can be scanned with authenticator apps. Additionally, eight 10-character backup codes are generated and hashed (SHA-256) during MFA enablement to allow account recovery if the primary TOTP device is unavailable.

7.5.7. CSRF protection

All forms are protected with CSRF tokens. Handling these tokens is done entirely by Django thanks to `@ensure_csrf` wrapper, as shown in Figure 34.

```
@api_view(["POST"])
@permission_classes([AllowAny])
@ensure_csrf_cookie
def login_view(request: Request):
    serializer = LoginSerializer(data=request.data)
```

Figure 34: Ensuring CSRF on forms with Django's built-in wrapper

7.5.8. CAPTCHA

To implement CAPTCHA I used Google's reCAPTCHA v2, which protects registration, login, forgot password, and resend activation endpoints by verifying the captcha token server-side through Google's verification API. Code snippet used for verifying the captcha token is shown in Figure 35.

```
secret_key = getattr(settings, 'RECAPTCHA_SECRET_KEY', None)

if not secret_key:
    logger.warning("RECAPTCHA_SECRET_KEY not configured, skipping verification")
    return True

try:
    response = requests.post(
        'https://www.google.com/recaptcha/api/siteverify',
        data={
            'secret': secret_key,
            'response': token,
        },
        timeout=10
    )
    result = response.json()

    if result.get('success'):
        return True
    else:
        logger.warning(f"reCAPTCHA verification failed: {result.get('error-codes', [])}")
        return False

except requests.RequestException as e:
    logger.error(f"reCAPTCHA verification request failed: {e}")
    return False
```

Figure 35: Code snippet used for verifying the captcha token

7.5.9. Session management

Session management is handled exclusively by Django using database-backed sessions. Sessions are created for authenticated users after successful MFA verification using the `login()` method and disposed using the `logout()` method. Sessions are configured with security best practices: `SESSION_COOKIE_HTTPONLY` prevents JavaScript access, `SESSION_COOKIE_SECURE` ensures HTTPS-only transmission, `SESSION_COOKIE_SAMESITE = "Lax"` provides CSRF protection, and sessions expire after 24 hours (`SESSION_COOKIE_AGE = 86400`).

7.5.10. Logging

I implemented flat-file logging in my Django backend to track events throughout the application, including security events and API calls. The logging system uses a `TimedRotatingFileHandler` that rotates log files daily at midnight, retaining the last 7 days of logs. All events are logged to both a file (`logs/app_sec.log`) and the

console for real-time monitoring during development. Different log levels are configured for various components: API requests are logged at the INFO level, account-related events at the DEBUG level, and critical Django request errors at the CRITICAL level. Part of the log file generated during development, showing security logs for registration and email confirmation (both successful and unsuccessful), is shown in Figure 36.

```
2025-12-07 16:15:06,530 - api.requests - INFO - POST /auth/register/ - Status: 201 - Duration: 1.159s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 16:16:55,915 - api.requests - INFO - GET /auth/confirm-email/ - Status: 200 - Duration: 0.017s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 16:39:03,529 - api.requests - WARNING - GET /auth/status/ - Status: 401 - Duration: 0.000s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 16:44:31,108 - api.requests - WARNING - GET /auth/status/ - Status: 401 - Duration: 0.000s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 16:46:42,611 - api.requests - WARNING - POST /auth/register/ - Status: 400 - Duration: 0.635s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 16:47:40,797 - api.requests - WARNING - POST /auth/register/ - Status: 400 - Duration: 0.633s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:25:18,502 - api.requests - WARNING - POST /auth/register/ - Status: 400 - Duration: 0.641s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:25:35,361 - api.requests - WARNING - POST /auth/register/ - Status: 400 - Duration: 0.645s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:27:07,831 - api.requests - WARNING - POST /auth/register/ - Status: 400 - Duration: 0.635s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:31:16,212 - api.requests - WARNING - GET /auth/confirm-email/ - Status: 406 - Duration: 0.003s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:31:17,683 - api.requests - WARNING - GET /auth/confirm-email/ - Status: 406 - Duration: 0.002s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:31:17,982 - api.requests - WARNING - GET /auth/confirm-email/ - Status: 406 - Duration: 0.003s - User: AnonymousUser - IP: 127.0.0.1
2025-12-07 17:31:18,140 - api.requests - WARNING - GET /auth/confirm-email/ - Status: 406 - Duration: 0.003s - User: AnonymousUser - IP: 127.0.0.1
```

Figure 36: Security logs generated during development

7.5.11. Sending emails

As mentioned, I use the PUT SMTP server to send emails using my personal account. The emails sent from Django are in both plaintext and HTML versions (an example of the HTML version is shown in Figure 16a). A code snippet of the method used for sending activation emails is shown in Figure 37. The exact same methodology is used for sending other emails.

```python
logger.info(f"Sending activation email")
try:
    msg = EmailMultiAlternatives(
        subject=subject,
        body=text_content,  # Plain text version
        from_email=settings.DEFAULT_FROM_EMAIL,
        to=[recipient_email],
    )
    msg.attach_alternative(html_content, "text/html")  # HTML version
    msg.send()

    logger.info(f"Successfully sent activation email")
except Exception as e:
    logger.error(f"Failed to send activation email: {e}")
    raise
```

Figure 37: Snippet of the method used for sending activation emails

7.5.12. Backend endpoints

The backend supports the following authentication and user management endpoints:

- Registration endpoints:
  - `POST /auth/register/`: expects `email`, `username`, `password`, and `captcha_token`. Creates a new user account and sends activation email.
  - `GET /auth/confirm-email/`: expects `token` query parameter. Activates user account via email confirmation token.
  - `POST /auth/resend-activation/`: expects `email`, and `captcha_token`. Resends activation email to inactive accounts.
- Authentication endpoints:
  - `POST /auth/login/`: expects `email`, `password`, and `captcha_token`. Verifies credentials and returns an `mfa_token`. Does not create session. Returns `mfa_setup_required: true` if user needs to set up MFA, or `mfa_required: true` if user needs to verify MFA code.
  - `POST /auth/logout/`: No parameters. Terminates user session.

- ◇ `GET /auth/status/`: No parameters. Checks if user is authenticated.
- Forgotten password endpoints:
  - ◇ `POST /auth/forgot-password/`: expects `email` and `captcha_token`. Sends password reset email.
  - ◇ `POST /auth/reset-password/`: expects `token` and `new_password`. Changes the password using token sent via email.
- Multi-factor authentication (MFA) endpoints:
  - ◇ `POST /auth/mfa/setup/`: expects `mfa_token`. Generates and returns QR code and secret for TOTP setup. Creates `mfa_secret` if not already set.
  - ◇ `POST /auth/mfa/enable/`: expects `mfa_token` and `code` (6-digit TOTP). Verifies TOTP code, enables MFA, generates and returns backup codes (shown once), and creates user session.
  - ◇ `POST /auth/mfa/verify/`: expects `mfa_token` and `code` (6-digit TOTP or 10-character backup code). Verifies code during login and creates user session on success. If backup code is used, it is consumed and returns remaining count.
- User profile endpoints:
  - ◇ `GET /auth/me/`: No parameters. Returns authenticated user's profile information.
  - ◇ `POST /auth/change-password/`: expects `current_password` and `new_password`. Changes password for authenticated user.

7.5.13. Additional notes

As my user model inherits from Django's `PermissionsMixin` to simplify the implementation of the admin user, there are additional tables automatically created by Django. Since I did not design them, and they are automatically linked to my User model in a many-to-many relationship (user ID to group ID, user ID to permission ID), I do not find it reasonable to describe them in this documentation. This functionality remains unused for now, aside from demonstration purposes in the admin panel, so I leave it out to avoid obscuring the main design.

8. Conclusions

During this laboratory exercise I designed and implemented an authentication module. The design covers the component's purpose, security assumptions, functional and non-functional requirements, system architecture and tech stack, database structure, and UML sequence diagrams. The features incorporated into the design include:

- a user registration form with local input validation.
- a user login form with local input validation.
- MFA enforcement, enrollment, and verification.
- CAPTCHA on forms (register, login, password reset)
- server-side session management (cookie-based).
- user logout mechanism with client- and server-side session disposal.
- password reset ("forgotten password") functionality.
- server-side input validation and error handling.
- secure storage via password hashing and salting.
- an account activation mechanism utilizing unique tokens sent via email.
- activation email resending mechanism.

Additional functionalities, besides the required ones, that I included in my authentication module are:

- advanced password policy,
- sending the email activation link via an SMTP server,
- domain restrictions for email addresses (whitelist),
- improved password hashing,
- activation token hardening,
- security event logging,
- enforcing HTTPS.
- CSRF protection for all forms,
- CAPTCHA,
- MFA.

The scope of these functionalities may be broadened in the future.